

RSA (Rivest–Shamir–Adleman) Algorithm

Aim

To implement the **RSA algorithm** for secure public-key encryption and decryption of messages using a pair of public and private keys.

Algorithm

Key Generation

Step 1: Start the program.

Step 2: Choose two prime numbers **p** and **q**.

Step 3: Compute $n = p \times q$

Step 4: Compute Euler's Totient Function $\phi(n) = (p - 1)(q - 1)$

Step 5: Choose a public exponent **e** such that $1 < e < \phi(n)$ And $\gcd(e, \phi(n)) = 1$

Step 6: Compute the private key **d** such that $d \times e \equiv 1 \pmod{\phi(n)}$

Step 7: The Public Key is (**e, n**).

Step 8: The Private Key is (**d, n**).

Encryption

Step 9: Read the plaintext message **M**.

Step 10: Encrypt using the public key. $C = M^e \pmod n$

where

- M = Plaintext
- C = Ciphertext

Decryption

Step 11: Decrypt using the private key. $M = C^d \pmod n$

Step 12: Display the original plaintext.

Step 13: Stop the program.

C Program for RSA

```
#include <stdio.h>
```

```
int gcd(int a, int b){
```

```
    while(b != 0) {
```

```
        int t = b;
```

```

        b = a % b;

        a = t; }

return a;}

long long power(long long base, long long exp, long long mod){
    long long result = 1;
    while(exp > 0){
        if(exp % 2 == 1)
            result = (result * base) % mod;
        base = (base * base) % mod;
        exp = exp / 2}
    return result;}

int main(){
    int p, q, e;
    long long n, phi;
    int d = 0;
    long long M, C, D;
    printf("Enter two prime numbers (p and q): ");
    scanf("%d%d", &p, &q);
    n = p * q;
    phi = (p - 1) * (q - 1);
    printf("Enter public key e: ");
    scanf("%d", &e);
    if(gcd(e, phi) != 1) {
        printf("Invalid Public Key\n");
        return 0;}
    for(int i = 1; i < phi; i++){
        if((i * e) % phi == 1){
            d = i;
            break;}}
    printf("Public Key (e,n) = (%d,%lld)\n", e, n);
    printf("Private Key (d,n) = (%d,%lld)\n", d, n);
    printf("Enter Plaintext: ");

```

```

scanf("%lld", &M);

C = power(M, e, n);

printf("Encrypted Ciphertext = %lld\n", C);

D = power(C, d, n);

printf("Decrypted Plaintext = %lld\n", D);

return 0;

}

```

OUTPUT :

The screenshot shows an IDE window titled "D:\CSA5112 - Cryptography\LAB PROGRAMS\RSA.cpp" with the following code:

```

45 printf("Enter public key e: ");
46 scanf("%d", &e);
47
48 if(gcd(e, phi) != 1)
49 {
50     printf("Invalid Public Key\n");
51     return 0;
52 }
53
54 // Find private key d
55 for(int i = 1; i < phi; i++)
56 {
57     if((i * e) % phi == 1)
58     {
59         d = i;
60         break;
61     }
62 }
63
64 printf("Public Key (e,n) = (%d,%d)\n", e, n);
65 printf("Private Key (d,n) = (%d,%d)\n", d, n);
66
67 printf("Enter Plaintext: ");
68 scanf("%lld", &M);
69
70 // Encryption
71 C = power(M, e, n);
72 printf("Encrypted Ciphertext = %lld\n", C);
73
74 // Decryption
75 D = power(C, d, n);
76 printf("Decrypted Plaintext = %lld\n", D);
77
78 return 0;
79

```

The output window shows the following execution results:

```

D:\CSA5112 - Cryptography\LAB PROGRAMS\RSA.cpp
Enter two prime numbers (p and q): 17 19
Enter public key e: 5
Public Key (e,n) = (5,323)
Private Key (d,n) = (173,323)
Enter Plaintext: 7
Encrypted Ciphertext = 11
Decrypted Plaintext = 7

-----
Process exited after 24.86 seconds with return value 0
Press any key to continue . . .

```

The status bar at the bottom indicates: Line: 79 Col: 2 Sel: 0 Lines: 79 Length: 1456 Insert Done parsing in 0.016 seconds